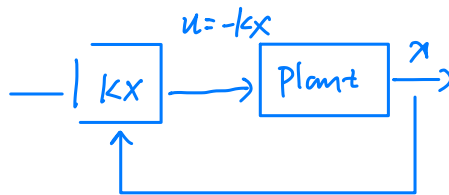


Lecture 4



$$\dot{x} = Ax + Bu$$

$$y = Cx + Du$$

$$\text{assume: } C = I \quad D = 0 \\ \text{so: } y = x$$

LQR and Continuous Dynamic Programming

In the previous lectures, we discussed dynamic programming for discrete states, $s \in \mathbb{S}$, and actions $a \in \mathbb{A}$. However, this is quite limiting, since in robotics we often have to reason about continuous state and action spaces.

4.1. Value Iteration on a Mesh

One way to extend value iteration to continuous state spaces is to do interpolation on a mesh. Here we assume we still discretize the action space and time.

What we would like to do is apply the value iteration update as follows:

$$\hat{J}^*(x) \leftarrow \min_a [g(x, a) + \gamma \hat{J}^*(f(x, a))] = \min_a [g(x, a) + \gamma \hat{J}^*(x')] \quad (4.1)$$

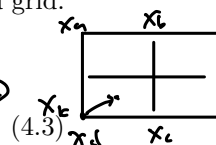
where

$$x_{k+1} = f(x_k, a_k). \quad (4.2)$$

However, due to the continuous nature of the state space, x_{k+1} may not be a point on a discretized grid. One solution to this problem is to use an interpolation scheme (e.g., barycentric interpolation).

Consider the interpolation strategy:

$$\hat{J}^*(x) = \sum_i \beta_i(x) \hat{J}^*(x_i), \quad \sum_i \beta_i = 1 \quad \beta_i > 0.$$



Here β_i is a weight on the i^{th} mesh point, x_i . Note that we are violating our typical notation where the subscript is not indicating time. Substituting this into our value iteration update gives

$$\hat{J}^*(x) \leftarrow \min_a [g(x, a) + \gamma \sum_i \beta_i(x') \hat{J}^*(x_i)], \quad (4.4)$$

or, in vector form

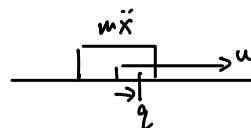
$$\vec{J}^* \leftarrow \min_a [\vec{g} + \gamma P \vec{J}^*]. \quad (4.5)$$

This final expression should look familiar!

4.2. Minimum-Time Control

Consider the simple double-integrator system

$$\ddot{q} = u,$$



$$(4.6)$$

with bounds on the input $|u| \leq 1$.

Suppose we want to solve for the minimum time policy

$$\min_{\pi} t_f \quad (4.7)$$

$$s.t. \quad q(t_0) = q_0, \quad (4.8)$$

$$q(t_f) = 0, \quad (4.9)$$

$$\ddot{q} = u \quad (4.10)$$

$$|u| \leq 1. \quad (4.11)$$

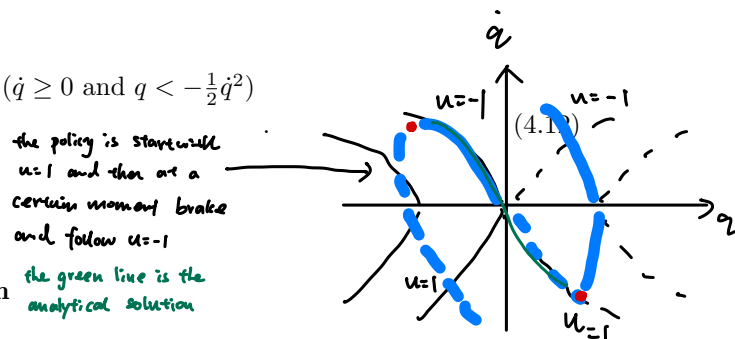
We end up with a bang-bang controller given as

$$\begin{cases} -1 & (\dot{q} < 0 \text{ and } q \leq \frac{1}{2}\dot{q}^2) \text{ or } (\dot{q} \geq 0 \text{ and } q < -\frac{1}{2}\dot{q}^2) \\ 0 & \text{if } q = 0 \wedge \dot{q} = 0 \\ -1 & \text{if otherwise} \end{cases}$$

For a detailed proof, see Kirk, Example 5.4-4.

4.3. Minimum-Time Control using Value Iteration

Can we approximate the minimum time solution using value iteration on mesh? See MATLAB example.



4.4. Discrete-Time Finite-Horizon LQR

With regards to our computational dynamic programming based approaches (e.g. value iteration), is there something better we can do in terms of handling continuous states and actions? There is, in the case of linear systems.

The original cost function (now rewritten for continuous state, x_k , and control, u_k) is given as

$$\min_{x_k, u_k} J = \sum_{k=0}^{N-1} g_k(x_k, u_k) + g_N(x_N) \quad (4.13)$$

$$s.t. \quad x_{k+1} = f(u_k, x_k) \quad i = 0, 1 \dots N-1. \quad (4.14)$$

Recall the Bellman Equation

$$J_k^*(x_k) = \min_{u_k \in U} [g(x_k, u_k) + J_{k+1}^*(f(x_k, u_k))]. \quad (4.15)$$

Assume

$$x_{k+1} = Ax_k + Bu_k \quad (4.16)$$

$$g(x_k, u_k) = x_k^T Q x_k + u_k^T R u_k \quad \text{quadratic cost} \quad (4.17)$$

$$g(x_N) = x_N^T Q_N x_N, \quad \text{or in } J = \int_0^\infty (x^T Q x + u^T R u) dt \quad (4.18)$$

where $Q \geq 0, Q_N \geq 0, R > 0$.

Here, we introduce the notion of a function being “positive-definite”. This notion first appears with regards to quadratic functions of the form $V(x) = x^T M x$.

A scalar continuous function $V(x)$ is said to be locally positive definite if

$$V(x) > 0, \quad \forall x \in \mathcal{D} - \{0\}, \quad V(0) = 0. \quad (4.19)$$

For functions, we will denote this $V(x) > 0, \quad \forall x \in \mathcal{D}$. If this holds over the whole space, $V(x)$ is globally positive definite.

A function $V(x)$ is locally negative definite if $-V(x)$ is locally positive definite. This will be denoted

$$\ddot{q} = u = -1$$

$$\dot{q}(t) = \dot{q}(0) - t \rightarrow t = \dot{q}(0) - q$$

$$q(t) = q(0) + \dot{q}(0)t - \frac{t^2}{2}$$

$$q(t) = \frac{1}{2}\dot{q}^2 + q(0) + \frac{\dot{q}(0)^2}{2}$$

$$u = +1$$

$$q(t) = \frac{1}{2}\dot{q}^2 + q(0) - \frac{1}{2}\dot{q}(0)^2$$

as $V(x) < 0, \quad \forall x \in D$.

A function $V(x)$ is locally positive semi-definite if

$$V(x) \geq 0, \quad \forall x \in \mathcal{D} - \{0\}, \quad V(0) = 0. \quad (4.20)$$

This will be denoted as $V(x) \geq 0, \quad \forall x \in D$. A function $V(x)$ is locally negative semi-definite if $-V(x)$ is locally positive semi-definite and will be denoted $V(x) \leq 0, \quad \forall x \in D$. The global properties follow as in the case of a positive definite function above.

For global positive definite quadratic functions of the form $V(x) = x^T M x > 0, \quad \forall x \in \mathbb{R}^n$, we sometimes refer to the matrix M as being a “positive definite matrix”. This is often denoted $M > 0$. Negative definite, positive semi-definite, and negative semi-definite matrices are represented as $M < 0$, $M \geq 0$, and $M \leq 0$ respectively.

Substituting in we have

$$J_k^*(x_k) = \min_{u_k} \left[\underbrace{x_k^T Q x_k + u_k^T R u_k}_{\mathcal{J}} + \underbrace{J_{k+1}^*(x_k, u_k)}_{\mathcal{J}_N^*} \right]. \quad (4.21)$$

Starting at the last time step $k = N$, we can write

$$J_N^*(x_N) = x_N^T Q_N x_N, \quad (4.22)$$

$$J_{N-1}^*(x_{N-1}) = \min_{u_{N-1}} \left[x_{N-1}^T Q_N x_{N-1} + u_{N-1}^T R u_{N-1} + x_N^T Q_N x_N \right] \quad (4.23)$$

and

$$J_{N-1}^*(x_{N-1}) = \min_{u_{N-1}} \left[x_{N-1}^T Q_N x_{N-1} + u_{N-1}^T R u_{N-1} + \right. \quad (4.24)$$

$$\left. (A_{N-1} x_{N-1} + B_{N-1} u_{N-1})^T Q_N (A_{N-1} x_{N-1} + B_{N-1} u_{N-1}) \right]. \quad (4.25)$$

Taking the derivative with respect to the control inputs gives

$$\frac{\partial J_{N-1}^*(x_{N-1})}{\partial u_{N-1}} = 2u_{N-1}^T R_{N-1} + 2(A_{N-1} x_{N-1} + B_{N-1} u_{N-1})^T Q_N B_{N-1}. = 0 \quad (4.26)$$

A useful identity for taking the partial derivative of the matrix quadratic form $x^T P x$ with respect to x can be derived as follows. Let $y = P x$, then

$$x^T P x = x^T y. \quad (4.27)$$

Since this is a scalar $x^T y = y^T x$. We can apply the product rule to take the partial derivative as

$$\frac{\partial}{\partial x} x^T y = y^T + x^T P \quad (4.28)$$

$$= x^T P^T + x^T P \quad (4.29)$$

$$= x^T (P^T + P). \quad (4.30)$$

If P is symmetric positive definite,

$$\frac{\partial}{\partial x} x^T P x = 2x^T P. \quad (4.31)$$

Setting the derivative equal to zero yields the control inputs resulting in the optimal cost-to-go given as

$$u_{N-1}^* = \left[-[R_{N-1} + B_{N-1}^T Q_N B_{N-1}]^{-1} B_{N-1}^T Q_N A_{N-1} \right] x_{N-1} \quad (4.32)$$

$$= -K_{N-1} x_{N-1}. \quad (4.33)$$

$\downarrow$
LQR

Now substitute back in the optimal policy into the dynamic programming recursion to get

$$J_{N-1}^*(x_{N-1}) = x_{N-1}^T Q x_{N-1} + x_{N-1}^T K_{N-1} R K_{N-1} x_{N-1} + \quad (4.34)$$

$$(A_{N-1} x_{N-1} - B_{N-1} K_{N-1} x_{N-1})^T Q_N (A_{N-1} x_{N-1} - B_{N-1} K_{N-1} x_{N-1}). \quad (4.35)$$

Finally, collect the x_{N-1} terms and write

$$J_{N-1}^*(x_{N-1}) = x_{N-1}^T (Q_{N-1} + K_{N-1}^T R K_{N-1} + \quad (4.36)$$

$$(A_{N-1} - B_{N-1} K_{N-1})^T Q_N (A_{N-1} - B_{N-1} K_{N-1})) x_{N-1} \quad (4.37)$$

$$= x_{N-1}^T S_{N-1} x_{N-1}. \quad (4.38)$$

Now we can assume and then verify that the same properties hold at $k-1$ and k and N and $N-1$. We have

$$u_k^* = -K_k x_k \quad (4.39)$$

and

$$J_k^*(x_k) = \frac{1}{2} x_k^T S_k x_k \quad (4.40)$$

where

$$S_k = Q + K_k^T R K_k + (A_k - B_k K_k)^T S_{k+1} (A_k - B_k K_k) \quad (4.41)$$

and

$$K_k = (R + B_k^T S_{k+1} B_k)^{-1} B_k^T S_{k+1} A_k. \quad (4.42)$$

4.5. Discrete-Time Infinite Horizon LQR

Riccati Equation

$$A^T P + P A - P B R^{-1} B^T P + Q = 0 \quad C P = P^T J$$

In the case of value iteration, we observed how the cost-to-go and the policy converged to a fixed-point. If A and B are not functions of time, K_k and S_k converge if the system is stabilizable. Dropping the time index, we have

$$S = Q + K^T R K + (A - B K)^T S (A - B K) \quad (4.43)$$

and

$$K = (R + B^T S B)^{-1} B^T S A. \quad (4.44)$$

4.6. Continuous-Time LQR

4.6.1 The Hamilton-Jacobi-Bellman Equation

We have seen the dynamic programming recursion

$$J_k^*(x_k) = \min_{u_k \in U} [g(x_k, u_k) + J_{k+1}^*(f(x_k, u_k))]. \quad (4.45)$$

This is useful for discrete time scenarios. However, if we take the limit as $t \rightarrow 0$, we arrive at the Hamilton-Jacobi-Bellman equation.

Divide the time-horizon in $\delta = T/N$ with $x_k = x(k\delta)$ and $u(k\delta)$ for $k = 0, 1, \dots, N$.

The continuous-time dynamics are approximated by

$$x_{k+1} = x_k + f(x_k, u_k) \delta \quad (4.46)$$

and the cost-function is approximated by

$$h(x_N) + \sum_{k=0}^{N-1} g(x_k, u_k) \delta. \quad (4.47)$$

cost of final state

Using the DP recursion, we have

$$\hat{J}(N\delta, x) = h(x) \quad (4.48)$$

$$\hat{J}(k\delta, x) = \min_{u \in U} [g(x, u)\delta + \hat{J}((k+1)\delta, x + f(x, u)\delta)]. \quad (4.49)$$

Do a Taylor series expansion of $\hat{J}$ around $(k\delta, x)$, and we have

$$\hat{J}((k+1)\delta, x + f(x, u)\delta) = \hat{J}(k\delta, x) + \frac{\partial \hat{J}}{\partial t} \delta + \frac{\partial \hat{J}}{\partial x} f(x, u)\delta + o(\delta). \quad (4.50)$$

Substituting in, we have

$$\hat{J}(k\delta, x) = \min_{u \in U} [g(x, u)\delta + \cancel{\hat{J}(k\delta, x)} + \frac{\partial \hat{J}}{\partial t} \delta + \frac{\partial \hat{J}}{\partial x} f(x, u)\delta + o(\delta)]. \quad (4.51)$$

higher term go to zero faster

If we divide by δ and let $\delta \rightarrow 0$, we get the HJB equations:

$$0 = \min_u [g(x, u) + \frac{\partial J^*}{\partial t} + \frac{\partial J^*}{\partial x} f(x, u)], \quad (4.52)$$

$$\pi^*(x) = \arg \min_u [g(x, u) + \frac{\partial J^*}{\partial t} + \frac{\partial J^*}{\partial x} f(x, u)] \quad (4.53)$$

4.7. Continuous-Time Finite Horizon LQR

Consider the linear, time-varying system

$$\dot{x} = A(t)x + B(t)u. \quad (4.54)$$

We desire to minimize the cost

$$J = g_f(x(t_f)) + \int_0^{t_f} g(x(t), u(t)) dt. \quad (4.55)$$

Here

$$g_f(x) = x^T Q_f x, \quad Q_f = Q_f^T \geq 0 \quad (4.56)$$

$$g(x, u) = x^T Q x + u^T R u, \quad Q = Q^T \geq 0, \quad R = R^T > 0. \quad (4.57)$$

Starting with the HJB

$$0 = \min_u \left[g(x, u) + \frac{\partial J^*}{\partial x} f(x, u) + \frac{\partial J^*}{\partial t} \right], \quad (4.58)$$

$$(4.59)$$

let's assume a particular form of the cost-to-go given as

$$J^*(x, t) = x^T S(t)x, \quad S(t) = S(t)^T \geq 0. \quad (4.60)$$

Substituting into the HJB, we have

$$0 = \min_u \left[g(x, u) + 2x^T S(t)f(x, u) + x^T \dot{S}(t)x \right], \quad (4.61)$$

$$= \min_u \left[x^T Q x + u^T R u + 2x^T S(t)(Ax + Bu) + x^T \dot{S}(t)x \right], \quad (4.62)$$

$$= \min_u \left[\underbrace{x^T Q x + \left(u^T R u \right) + 2x^T S(t)Ax + \left(2x^T S(t)Bu \right)}_{\text{du}} + x^T \dot{S}(t)x \right] \quad (4.63)$$

Now take the minimization with respect to u and set equal to zero as follows:

$$2u^T R + 2x^T S(t)B = 0 \quad (4.64)$$

$$Ru + B^T S(t)x = 0. \quad (4.65)$$

Solving for u gives

$$u = -R^{-1}B^T S(t)x \quad (4.66)$$

$$= -Kx. \quad (4.67)$$

Substitute in for u and get

$$x^T Qx + x^T K^T R K x + 2x^T S(t)Ax - 2x^T S(t)BKx + x^T \dot{S}(t)x = 0, \quad (4.68)$$

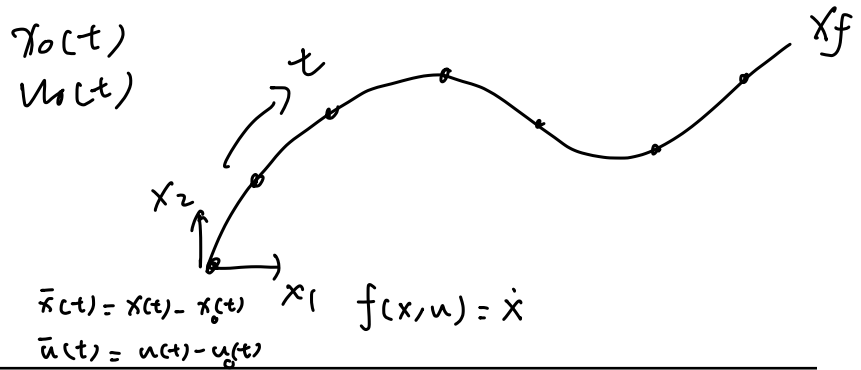
$$Q + K^T R K + 2S(t)A + 2S(t)BK = -\dot{S}(t), \quad (4.69)$$

and

$$S(t_f) = Q_f. \quad (4.70)$$

Bibliography

- [1] Russ Tedrake. *Underactuated Robotics*. 2023.
- [2] Dimitri Bertsekas. *Reinforcement Learning and Optimal Control*. Athena Scientific, 2019.
- [3] Drew Bagnell, Byron Boots, and Sanjiban Choudhury. *Modern Adaptive Control and Reinforcement Learning*. 2022.



Lecture 5

Iterative LQR and Differential Dynamic Programming

We have seen that we can apply dynamic programming numerically through value iteration to achieve control of a large class of nonlinear systems subject to dimensionality constraints. We have also been able to apply dynamic programming analytically to solve the special case of linear time-invariant systems.

However, we can in fact, apply dynamic programming to nonlinear systems, if we are willing to find a solution that is only locally valid.

5.1. Time-Varying LQR for Trajectory Tracking

Assume we have a nominal trajectory given by $x_0(t)$ and $u_0(t)$.

Define a local coordinate system relative to that trajectory where $\bar{x}(t) = x(t) - x_0(t)$ and $\bar{u}(t) = u(t) - u_0(t)$. We can then write

$$\dot{\bar{x}} = \dot{x} - \dot{x}_0 = f(x, u) - f(x_0, u_0) \quad (5.1)$$

$$\dot{\bar{x}} \approx f(x_0, u_0) + \frac{\partial f(x_0, u_0)}{\partial x}(x - x_0) + \frac{\partial f(x_0, u_0)}{\partial u}(u - u_0) - f(x_0, u_0) \quad (5.2)$$

$$\approx A(t)\bar{x} + B(t)\bar{u}. \quad (5.3)$$

Then use LQR to compute $K(t)$.

$$u = -K(t)x$$

Therefore, if we have a feasible trajectory, we can easily compute a time-varying feedback policy to achieve tracking of that trajectory.

The linearization required may seem limiting, but in practice it is really quite powerful and can provide a means of controlling fairly challenging underactuated systems.

5.2. Iterative LQR

Still, it would be nice if we could apply the machinery of dynamic programming to arbitrary optimal control problems. It turns out that there is a way to use the principles of LQR (linearizing about a trajectory) to solve for both the feedback policy and the trajectory.

Consider the value function, V_k and associated Q-function, Q_k . Here

$$V_k = \min_u [g(x_k, u_k) + V_{k+1}(f(x_k, u_k))] \quad (5.4)$$

$$V_k = \min_u Q_k(x_k, u_k), \quad (5.5)$$

where $Q_k = g(x_k, u_k) + V_{k+1}(f(x_k, u_k))$.

Approximate Q_k to second order about a nominal trajectory. Let $\delta x = x - x^i$ and $\delta u = u - u^i$.

To start, we take the second order expansion of $g(x_k, u_k)$ about x_k^i and u_k^i .

$$g(x_k, u_k) = g(x_k^i, u_k^i) + \frac{\partial g}{\partial x_k} \delta x_k + \frac{\partial g}{\partial u_k} \delta u_k + \frac{1}{2} \delta x_k^T \frac{\partial^2 g}{\partial x_k^2} \delta x_k + \frac{1}{2} \delta u_k^T \frac{\partial^2 g}{\partial u_k^2} \delta u_k + \delta u_k^T \frac{\partial^2 g}{\partial u_k \partial x_k} \delta x_k \quad (5.6)$$

$$= g(x_k^i, u_k^i) + g_{x,k}^T \delta x_k + g_{u,k}^T \delta u_k + \frac{1}{2} \delta x_k^T g_{xx,k} \delta x_k + \frac{1}{2} \delta u_k^T g_{uu,k} \delta u_k + \delta u_k^T g_{ux,k} \delta x_k \quad (5.7)$$

where we assume $g_{xu,k} = g_{ux,k}^T$.

We then take the second order expansion of V_{k+1} , which is given as

$$V_{k+1} \approx V_{k+1}(x_k^i, u_k^i) + \frac{\partial V_{k+1}}{\partial x_{k+1}} \delta x_{k+1} + \frac{1}{2} \delta x_{k+1}^T \frac{\partial^2 V_{k+1}}{\partial x_{k+1}^2} \delta x_{k+1} \quad (5.8)$$

$$= V_{k+1}(x_k^i, u_k^i) + V_{x,k+1}^T \delta x_{k+1} + \frac{1}{2} \delta x_{k+1}^T V_{xx,k+1} \delta x_{k+1}. \quad (5.9)$$

where

$$\delta x_{k+1} = x_{k+1} - x_{k+1}^i \quad (5.10)$$

$$= A_k \delta x_k + B_k \delta u_k. \quad (5.11)$$

We can then write the second order Taylor series expansion of Q_k as

$$Q_k \approx Q_k(x_k^i, u_k^i) + \begin{bmatrix} Q_{x,k} \\ Q_{u,k} \end{bmatrix}^T \begin{bmatrix} \delta x_k \\ \delta u_k \end{bmatrix} + \frac{1}{2} \begin{bmatrix} \delta x_k \\ \delta u_k \end{bmatrix}^T \begin{bmatrix} Q_{xx,k} & Q_{ux,k}^T \\ Q_{ux,k} & Q_{uu,k} \end{bmatrix} \begin{bmatrix} \delta x_k \\ \delta u_k \end{bmatrix}. \quad (5.12)$$

For the Q-function, we know

$$\delta u = \arg \min_{\delta u} Q_k. \quad (5.13)$$

Taking the derivative and setting to zero gives

$$0 = Q_u^T + Q_{ux} \delta x + Q_{uu} \delta u. \quad (5.14)$$

The control δu can be given as

$$\delta u^* = -Q_{uu}^{-1} [Q_u^T + Q_{ux} \delta x] \quad (5.15)$$

$$= k + K \delta x. \quad (5.16)$$

Substitute this back into the Bellman equation gives

$$V_k(x_k^i, u_k^i) + V_{x,k}^T \delta x_k + \frac{1}{2} \delta x_k^T V_{xx,k} \delta x_k = Q_k(x_k^i, u_k^i) + \begin{bmatrix} Q_{x,k} \\ Q_{u,k} \end{bmatrix}^T \begin{bmatrix} \delta x_k \\ \delta u_k^* \end{bmatrix} + \frac{1}{2} \begin{bmatrix} \delta x_k \\ \delta u_k^* \end{bmatrix}^T \begin{bmatrix} Q_{xx,k} & Q_{ux,k}^T \\ Q_{ux,k} & Q_{uu,k} \end{bmatrix} \begin{bmatrix} \delta x_k \\ \delta u_k^* \end{bmatrix}. \quad (5.17)$$

This provides us with the backwards pass and allows us to compute $V_{x,k}, V_{xx,k}$.

The general algorithm for iterative-LQR is as follows:

Initialize with $k^i = 0$, $K^i = 0$, and a given u_k^i . *(initialize policy and input sequence)*

Iterate until convergence:

1. Forward Pass *(take the input u and run out policy)*

- Generate x_k^{i+1} and u_k^{i+1} using $x_{k+1}^{i+1} = f_k(x_k, u_k)$ where $u_k^{i+1} = u_k^i + \alpha k_k^i + K_k^i \delta x^i$

2. Backward Pass *(do all bellman update, linearize dynamic, get A, B, use algo to compute $V_{x,k}$ $V_{xx,k}$)*

- Linearize the dynamics about x_k^{i+1} and u_k^{i+1} to get A_k^{i+1} and B_k^{i+1} .
- Use the backwards pass to compute $V_{x,k}, V_{xx,k}$ from $V_{x,N} = g_{x,N}$ and $V_{xx,N} = g_{xx,N}$.
- Use $V_{x,k}, V_{xx,k}$ with K^{i+1} and k^{i+1} .

α can be chosen using the Armijo rule. *(use smaller step to update) $\rightarrow u_k^{i+1} = u_k^i + \alpha k_k^i + K_k^i \delta x^i$*

(update the policy and reiterate again)

5.3. Differential Dynamic Programming

Differential dynamic programming (DDP) is identical to DDP with the exception that a second order expansion of the dynamics is used, while the Q function is still truncated to second order.

Bibliography

- [1] Russ Tedrake. *Underactuated Robotics*. 2023.
- [2] Dimitri Bertsekas. *Reinforcement Learning and Optimal Control*. Athena Scientific, 2019.
- [3] Drew Bagnell, Byron Boots, and Sanjiban Choudhury. *Modern Adaptive Control and Reinforcement Learning*. 2022.

